Seguridad en Sistemas, Aplicaciones y el Big Data

Tema 2. Seguridad en aplicaciones web. Introducción

Índice

Esquema

Ideas clave

2.1. Introducción y objetivos

2.2. Arquitecturas de las aplicaciones web

2.3. Protocolo HTTP

2.4. Ciclo de vida de desarrollo seguro de software (SSDLC)

2.5. Referencias bibliográficas

A fondo

Descripción del modelo de arquitectura de aplicaciones SPA AJAX

Arquitectura de microservicios

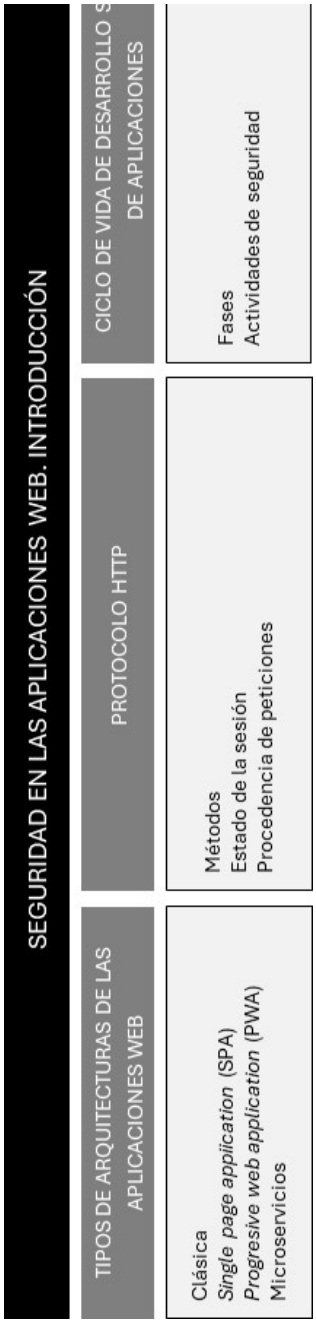
Mean stack

Patrón de diseño MVVM

The Application of a new secure software development Life Cycle (S-SDLC) with agile methodologies

Test

Esquema



© Universidad Internacional de La Rioja (UNIR)

2.1. Introducción y objetivos

El desarrollo web ha evolucionado significativamente en los últimos años y dio lugar a distintas arquitecturas que responden a necesidades específicas de rendimiento, escalabilidad, experiencia de usuario y mantenimiento. A continuación, se introducen los principales tipos de arquitecturas web utilizadas en la actualidad:

- **Arquitectura web clásica** (*multi-page application* o MPA). Es el modelo tradicional de las primeras aplicaciones web. En esta arquitectura, cada vez que el usuario realiza una acción (como hacer clic en un enlace), se recarga toda la página desde el servidor. El servidor genera el HTML completo y lo envía al navegador. Se utiliza ampliamente en sitios web informativos o aplicaciones simples.

- **SPA** *(single page application).* Una SPA carga una sola página HTML y actualiza dinámicamente el contenido sin recargar la página entera. Usa JavaScript (normalmente con *frameworks,* como React, Angular o Vue) para manejar las interacciones del usuario. Funcionamiento:

  - Al inicio, se carga una página HTML principal.
  - Luego, solo se intercambian datos con el servidor (generalmente vía API REST o GraphQL).

- **PWA***(progressive web application).* Una PWA combina lo mejor de las aplicaciones web y móviles. Son sitios web que se comportan como aplicaciones nativas en dispositivos móviles. Características clave:

  - Funcionan *offline* gracias a los *service workers*.
  - Se pueden instalar en el dispositivo sin pasar por una tienda de aplicaciones.
  - Responden rápidamente y son *responsive* (adaptables a diferentes tamaños de pantalla).

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Arquitectura de microservicios.** En lugar de una única aplicación grande (monolito), se divide en pequeños servicios independientes que se comunican entre sí. Características:

  - Cada microservicio es responsable de una funcionalidad específica (por ejemplo, usuarios, pagos, productos).
  - Se comunican, generalmente, mediante API (REST o gRPC).
  - Se pueden desarrollar, escalar y desplegar por separado.

Este tema, además, analiza el protocolo HTTP, que es el mecanismo estándar para la comunicación en la web, que se enfoca en algunos problemas de seguridad debido a su diseño y sus métodos. Funciona sobre el protocolo TCP/IP y permite que los navegadores (clientes) soliciten recursos —como páginas HTML, imágenes o datos— a los servidores.

Características clave de HTTP:

- **Protocolo sin estado**: cada solicitud es independiente; el servidor no recuerda solicitudes anteriores del mismo cliente (aunque esto puede manejarse con *cookies* o sesiones).
- **Métodos HTTP**: los más comunes incluyen GET (obtener datos), POST (enviar datos), PUT (actualizar recursos) y DELETE (eliminar recursos).
- **HTTP/1.1, HTTP/2 y HTTP/3**: cada nueva versión del protocolo mejora el rendimiento, la seguridad y la eficiencia de las comunicaciones.

HTTP es fundamental para el funcionamiento de las arquitecturas web modernas, ya que actúa como el puente que permite la interacción entre el cliente y el servidor y facilita la entrega dinámica de contenidos y servicios en la web.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Para diseñar, implementar y probar la seguridad en las aplicaciones desde las primeras fases de su construcción, se debe adoptar un **ciclo de vida de desarrollo seguro de *software* (SSDLC)** que permita acometer el diseño e implementación de la seguridad desde las primeras fases del ciclo de vida de desarrollo de aplicaciones.

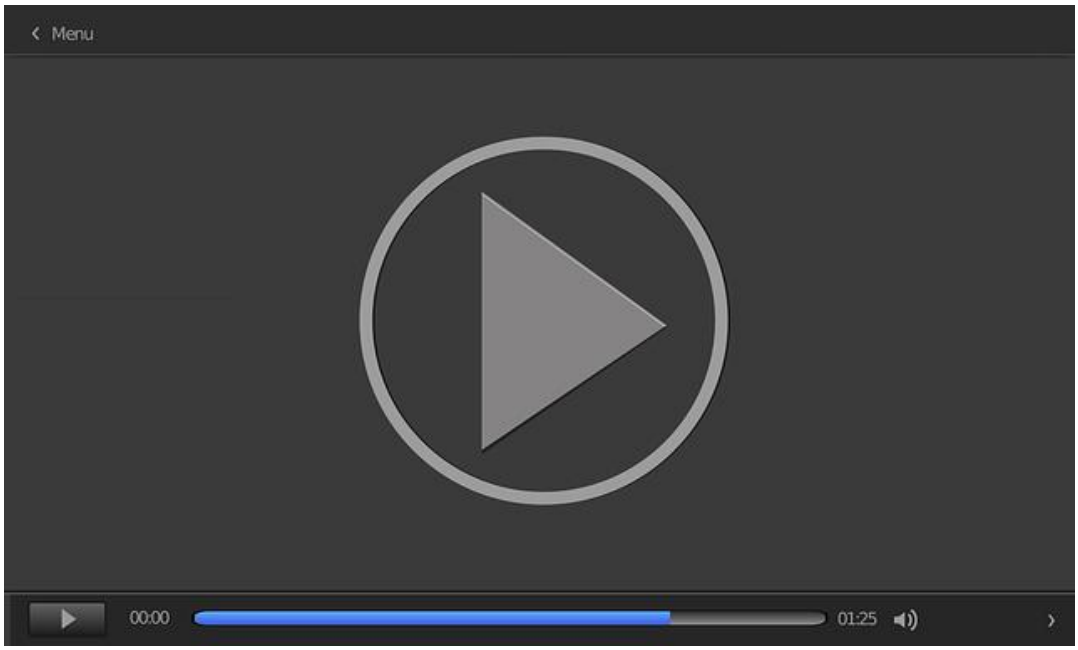
El proceso **SSDLC** incluye una serie de actividades de seguridad que implementar en cada una de sus fases, como pueden ser el análisis y gestión del riesgo de forma dinámica, la revisión de seguridad del código, las pruebas de seguridad o la monitorización continua en fase de despliegue.

Las aplicaciones web son por excelencia el principal tipo de aplicación *online* accesible desde Internet, las Intranets de organizaciones, mediante una PC, una Tablet o un móvil, características que también posibilitan que sean accesibles por posibles atacantes que quieren obtener algún beneficio económico, información, etc. Por tanto, es necesario introducir las características de las principales **arquitecturas de aplicaciones web** y de su **protocolo HTTP** empleado en la comunicación.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

En el vídeo *Proyectos de seguridad web* repasaremos los principales proyectos que se ocupan de la seguridad de las aplicaciones web.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=4e7f4317-2882-46f2-a383-adbf010b1f48

Los **principales objetivos** son los siguientes:

- Analizar los distintos tipos de arquitecturas de aplicaciones web.
- Analizar el protocolo HTTP y sus problemas relacionados con la seguridad.
- Aprender qué actividades de seguridad hay que llevar a cabo en el ciclo de vida de desarrollo seguro (SSDLC).

© Universidad Internacional de La Rioja (UNIR)

2.2. Arquitecturas de las aplicaciones web

El extraordinario éxito del modelo web puede atribuirse a una característica fundamental: es un modelo que se acopla más débilmente que los modelos de programación distribuida tradicionales como RPC, RMI, DCOM y CORBA. Las interacciones entre clientes y servidores web son sencillas: intercambian mensajes que transportan datos de tipo MIME y la semántica de un mensaje puede modificarse utilizando cabeceras. El destino de un mensaje se especifica indirectamente utilizando una URL y este nivel de indirección puede aprovecharse para implementar balance de carga, seguimiento de sesiones y otras funcionalidades. Las **arquitecturas** de **aplicaciones web** actualmente son:

- Clásica monolítica. *Multiple page application* (MPA)
- *Single page application* (SPA)
- Progresivas (PWA)
- Servicios web (REST, SOAP, GRADLE, etc.)

Arquitectura web clásica monolítica

Las **arquitecturas monolíticas** son las que se utilizan normalmente cuando se desarrolla la aplicación mediante un lenguaje de *script,* como Coldfusion o PHP alojada en un servidor web.

Cuando se necesita una aplicación **escalable** y que ofrezca mayor **rendimiento** y **seguridad** se ha de recurrir a una de las opciones de desarrollo empresarial como las analizadas anteriormente: «.Net» o «J2EE» (Padilla and Pérez, 2010). Estas especificaciones proporcionarán mayores posibilidades debidas a las distintas opciones de **configuración e implementación de la seguridad** que ofrecen, por ejemplo, la utilización de lenguajes potencialmente más seguros como Java o C#.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Una arquitectura de aplicación escalable normalmente se divide en niveles y si se utilizan patrones de diseño. Muchas veces se dividen en porciones reutilizables, usando diferentes lineamientos específicos para reforzar su modularidad, requerimientos de interfaz y la reutilización de objetos.

Además, dividir la aplicación en niveles permite que esta se pueda distribuir entre varios servidores, mejorando, por lo tanto, su escalabilidad a expensas de mayor complejidad con **controles de acceso y detección avanzados.** Existe separación de tareas y responsabilidades.

Características principales:

- **Monolítico:** toda la aplicación se implementa como una sola unidad indivisible. Esto significa que todo el código fuente se compila y empaqueta en un único ejecutable o archivo de despliegue.

- **Capas** o *tiers* (Figura 2): generalmente, la arquitectura monolítica se organiza en capas lógicas para separar las responsabilidades. Las más comunes son:

  - **Capa de presentación *(frontend):*** esta es la capa con la que el usuario final interactúa directamente. Su principal función es mostrar la interfaz de usuario y capturar las entradas del usuario. Componentes: páginas web (HTML, CSS, JavaScript), interfaces de escritorio o aplicaciones móviles. Función: traduce las interacciones del usuario en peticiones que se envían a la capa de lógica de negocio.

  - **Capa de lógica de negocio:** es el corazón de la aplicación. Contiene las reglas, la lógica y los procesos que definen cómo funciona el negocio. Componentes: módulos que procesan las peticiones, validan datos, realizan cálculos y orquestan la interacción con la capa de datos. Función: procesa las peticiones recibidas de la capa de presentación y decide qué acciones tomar.

© Universidad Internacional de La Rioja (UNIR)

- **Capa de acceso a datos:** es el corazón de la aplicación. Contiene las reglas, la lógica y los procesos que definen cómo funciona el negocio. Componentes: módulos que procesan las peticiones, validan datos, realizan cálculos y orquestan la interacción con la capa de datos. Función: procesa las peticiones recibidas de la capa de presentación y decide qué acciones tomar.

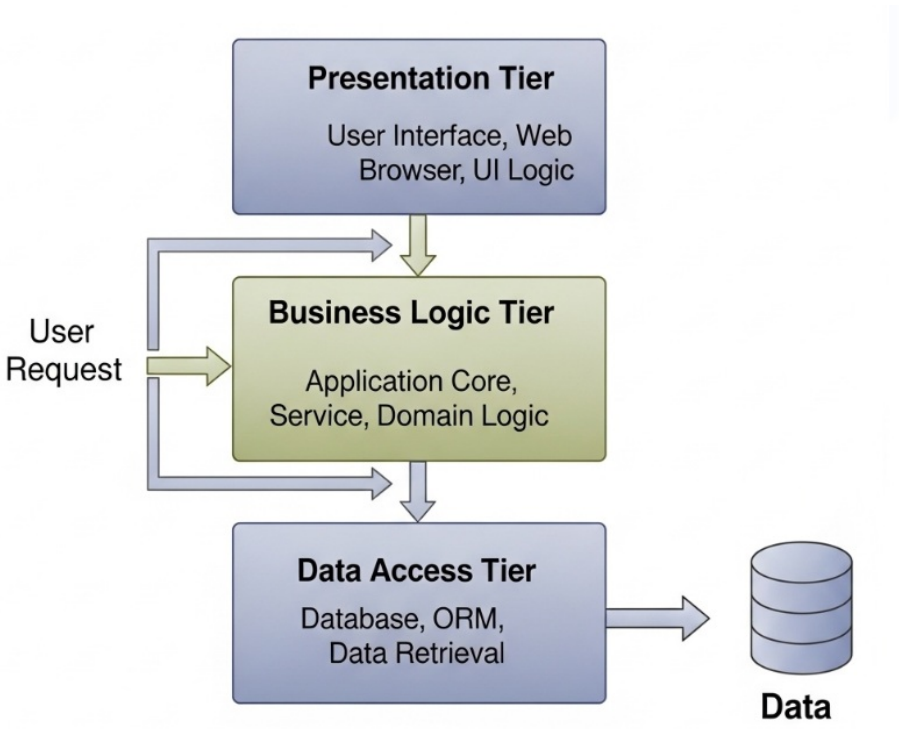


Figura 1. Arquitectura de tres capas de aplicaciones web. Fuente: elaboración propia.

Arquitectura web SPA AJAX

Una aplicación SPA *(single-page application)* es una arquitectura web donde una aplicación carga una sola página HTML y luego **actualiza dinámicamente** el **contenido** a medida que el usuario interactúa con ella, en lugar de recargar la página completa. El término AJAX (Asynchronous JavaScript and XML) (Mayekar, 2018), se refiere a la técnica subyacente que permite esta funcionalidad, haciendo peticiones al servidor en segundo plano sin interrumpir la experiencia del usuario.

© Universidad Internacional de La Rioja (UNIR)

Características clave:

- **Carga única de la página:** el navegador descarga inicialmente un solo archivo HTML junto con el JavaScript y CSS necesarios para la aplicación.
- **Actualizaciones dinámicas:** cuando el usuario navega o realiza una acción, la aplicación no recarga la página. En cambio, usa AJAX para enviar y recibir datos del servidor.
- **Comunicación asíncrona:** AJAX permite que la aplicación solicite datos del servidor en segundo plano sin congelar la interfaz de usuario. El servidor responde con datos (generalmente, en formato JSON), que el JavaScript del cliente utiliza para actualizar selectivamente partes de la página.
- **Separación *frontend/backend:*** esta arquitectura promueve una clara división entre la capa de presentación (el SPA en el navegador) y la capa de lógica de negocio (el *backend*). El *frontend* es responsable de la interfaz y la experiencia del usuario, mientras que el *backend* se enfoca en proporcionar una API para manejar datos y lógica de negocio.
- ***Frameworks:*** Angular, React, Vue, etc.

Cómo funciona:

- **Carga inicial:** el usuario accede a la URL y el servidor envía el archivo HTML principal junto con los archivos JavaScript y CSS.
- **Solicitud AJAX:** el usuario hace clic en un enlace o botón. En lugar de navegar a una nueva URL, el JavaScript intercepta el evento y realiza una petición asíncrona al servidor.
- **Respuesta del servidor:** el servidor procesa la petición y responde con un paquete de datos ligeros (normalmente JSON), no con una página HTML completa.

© Universidad Internacional de La Rioja (UNIR)

- **Actualización del DOM:** el JavaScript de la aplicación en el cliente recibe los datos y utiliza esa información para actualizar el DOM (el árbol de la página HTML) y modificar solo la parte de la interfaz que necesita ser cambiada.

Ventajas:

- **Mejor experiencia de usuario:** la navegación es más rápida y fluida, similar a una aplicación de escritorio, ya que no hay recargas de página.
- **Menor carga en el servidor:** después de la carga inicial, el servidor solo necesita enviar datos y no el HTML completo, lo que reduce la carga de procesamiento y el ancho de banda.
- **Desarrollo independiente:** el *frontend* y el *backend* pueden ser desarrollados por equipos separados de forma más independiente.

Desventajas:

- **Complejidad del lado del cliente:** la lógica de la aplicación se traslada, en gran medida, al cliente, lo que puede hacer que el código JavaScript sea más complejo y difícil de mantener.
- **Carga inicial lenta:** si el SPA es muy grande, la carga inicial del JavaScript puede ser pesada y tardar más tiempo.

El objetivo principal de las aplicaciones AJAX es eliminar la naturaleza *start-stop-start-stop* de la interacción entre el cliente y el servidor de aplicaciones web introduciendo un intermediario: un motor AJAX/JS entre el usuario y el servidor, que intercambia información con el servidor en el *backend* sin que el usuario lo perciba (como ocurre en la arquitectura web tradicional síncrona).

© Universidad Internacional de La Rioja (UNIR)

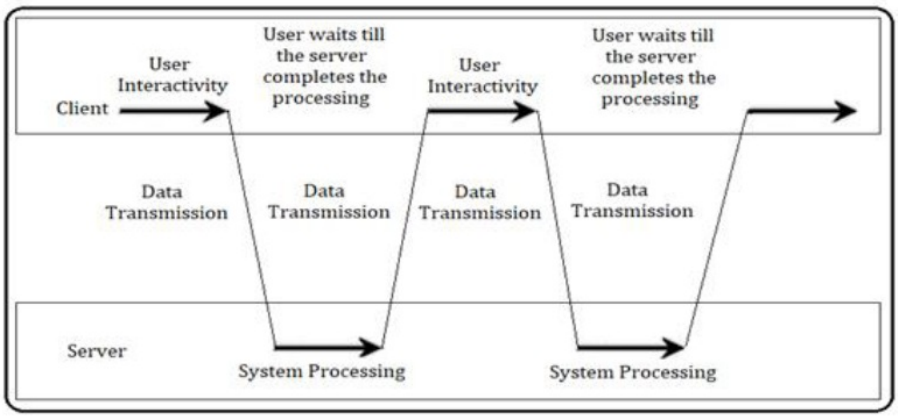


Figura 2. Modelo de comunicación síncrono. Fuente: Mayekar, 2018.

Este elemento arquitectónico adicional del lado del cliente es lo más característico de AJAX y tendrá algún inconveniente de seguridad, como veremos más adelante. El navegador cargará el motor AJAX usualmente escondido en un *frame* oculto y este será responsable del procesamiento tanto de la interfaz que el usuario ve, como de la comunicación con el servidor en nombre del usuario. Permitirá la interacción del usuario con la aplicación de forma asincrónica independientemente de la comunicación con el servidor.

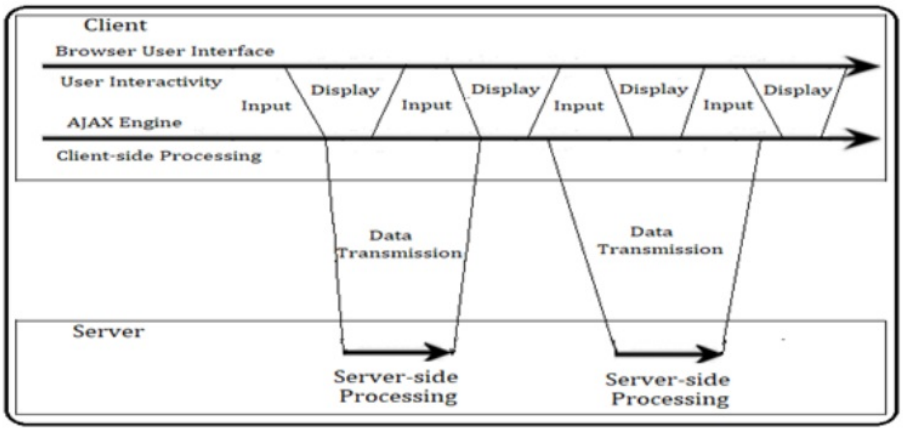


Figura 3. Modelo de comunicación asíncrono (AJAX). Fuente: Mayekar, 2018.

© Universidad Internacional de La Rioja (UNIR)

De esta forma, el usuario no queda esperando la respuesta del servidor, que tiene la percepción de que no hay tiempos de recarga. Los ***toolkits* AJAX** han permitido a los desarrolladores web construir aplicaciones web 2.0 de forma rápida y con, relativamente, poco esfuerzo, generando un nuevo nivel de interactividad con los usuarios como el que otorgan, por ejemplo: Google Maps, Google Docs, Flickr, etc.

En cuanto a la arquitectura en sí, se puede observar en la Figura 5. Cada acción del usuario que normalmente generaría una petición HTTP se convierte en una llamada JS al motor AJAX. El motor se encarga por sí solo de servir el código HTML, CSS y JS, ante cualquier respuesta a una acción del usuario que no requiere un viaje de vuelta al servidor, tales como: la validación de datos simple, la edición de datos en la memoria e incluso un poco de navegación. Si el motor necesita algo del servidor con el fin de responderle al cliente web, se encarga de realizar esas peticiones de forma asíncrona (HTTP Request), normalmente usando XML de modo transparente y sin penalizar *a priori* la experiencia del usuario del navegador (Asadullah et al. 2020).

© Universidad Internacional de La Rioja (UNIR)

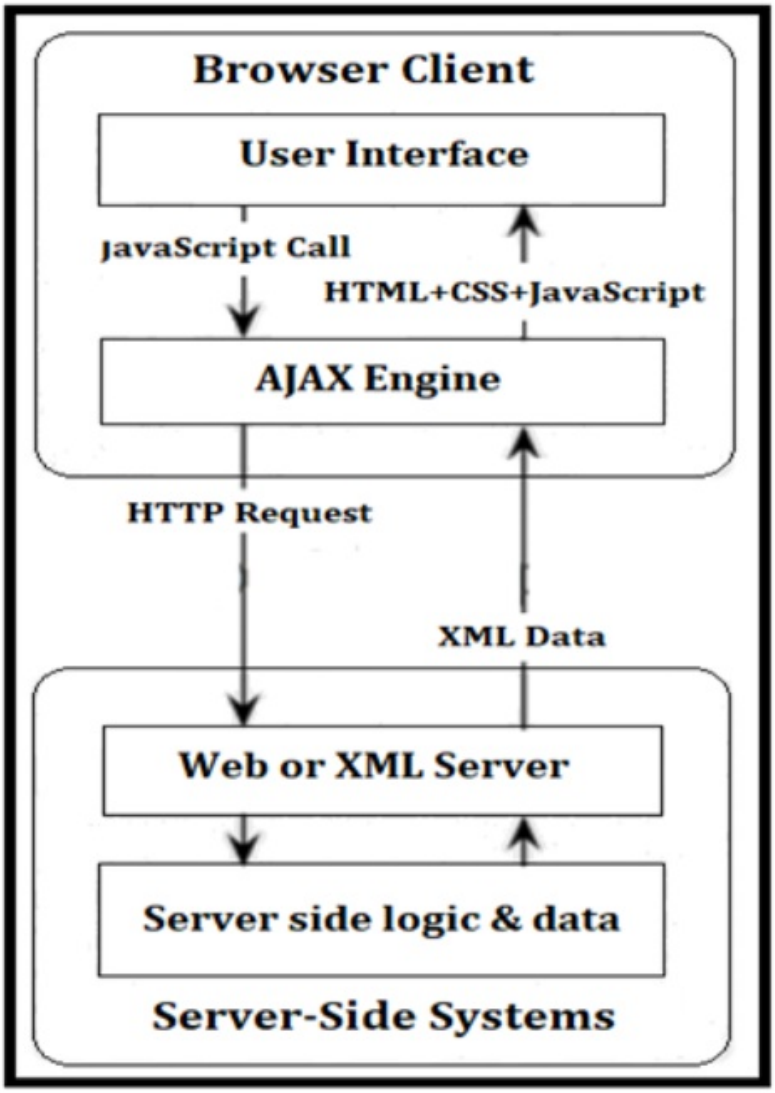


Figura 4. Modelo de aplicación AJAX web. Mayekar, 2018.

Arquitectura web progresiva PWA

Una arquitectura de aplicación web progresiva (PWA) es un enfoque de desarrollo que combina las características de las **aplicaciones móviles** y las aplicaciones **web tradicionales.** Su principal objetivo es ofrecer una experiencia de usuario **fluida** y de **alto rendimiento,** similar a la de una aplicación nativa, directamente desde el navegador web. Las PWA no son una tecnología única, sino una combinación de principios de diseño y API de navegador modernas (Wang, 2025).

© Universidad Internacional de La Rioja (UNIR)

Características clave de una PWA

- **Responsiva:** se adapta a cualquier dispositivo y tamaño de pantalla, ya sea un móvil, una *tablet* o un ordenador.
- **Independiente de la conectividad:** gracias a los *service workers,* pueden funcionar sin conexión o con una red deficiente porque almacenan en caché recursos esenciales y datos.
- **Instalable:** permite a los usuarios añadir la aplicación a la pantalla de inicio de su dispositivo, como si fuera una aplicación nativa.
- **Actualizable:** las actualizaciones se realizan de forma automática y transparente para el usuario y sin necesidad de visitar una tienda de aplicaciones.
- **Transporte seguro HTTPS:** requieren el uso de HTTPS, lo que garantiza la seguridad de la comunicación entre el cliente y el servidor.
- **Descubrible:** son detectables por los motores de búsqueda, a diferencia de las aplicaciones nativas.
- **Reenganchable:** pueden enviar notificaciones *push* a los usuarios para mantenerlos informados y fomentar su participación.

Componentes técnicos principales:

Los **elementos técnicos** que hacen posible las PWA son:

- ***Service workers:*** son *scripts* que se ejecutan en segundo plano, independientes de la página web. Actúan como un *proxy* programable entre el navegador y la red que les permite gestionar la caché y las notificaciones *push.*

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Manifiesto de la aplicación web** *(web app manifest):* es un archivo JSON que proporciona información sobre la aplicación (nombre, iconos, colores, orientación de la pantalla, etc.). Es lo que permite que la PWA sea instalable en la pantalla de inicio del dispositivo.

- **HTTPS:** el uso de un certificado SSL/TLS es un requisito fundamental para habilitar los *service workers* y asegurar que los datos no sean manipulados.

- *Frameworks:* Vue, React, AngularJS, Ionic, etc.

En esencia, las PWA ofrecen lo mejor de ambos mundos: la accesibilidad y el bajo coste de desarrollo de una aplicación web, con la velocidad, la capacidad de uso sin conexión y la experiencia de usuario de una aplicación nativa.

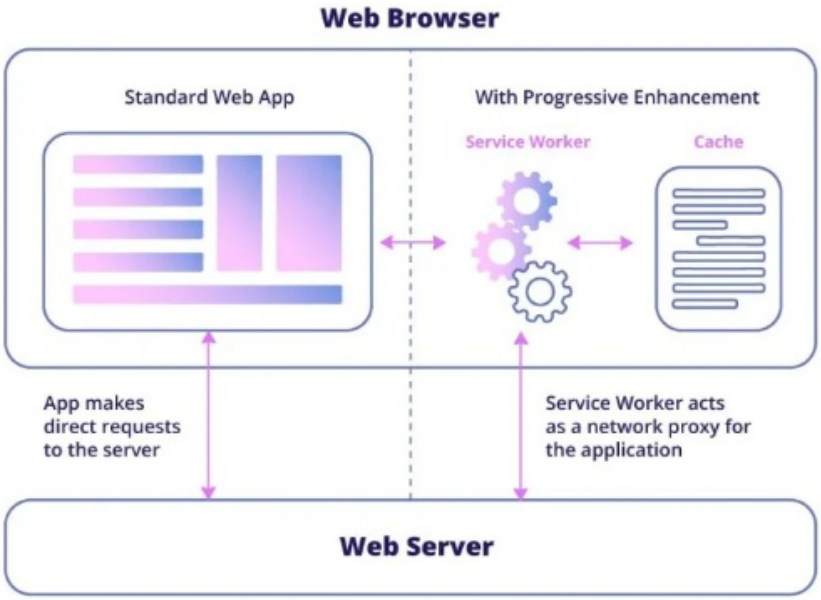


Figura 5. Arquitectura aplicaciones web PWA. Fuente: Nguyen, 2024.

© Universidad Internacional de La Rioja (UNIR)

Arquitectura de microservicios

La arquitectura de microservicios es un estilo de desarrollo de *software* que estructura una aplicación como una colección de **servicios pequeños** e **independientes.** Para ampliar cómo diseñar e implementar la seguridad en microservicios, pueden consultarse los estudios de Haindi (2024) y El Akhdar (2024).

Principios fundamentales:

- **Servicios pequeños y enfocados:** cada microservicio se centra en una única funcionalidad. Por ejemplo, en una aplicación de comercio electrónico podría haber un microservicio para la gestión de productos, otro para el carrito de compras y otro para el procesamiento de pagos.
- **Independencia tecnológica:** los equipos pueden elegir la tecnología (lenguaje de programación, base de datos) que mejor se adapte a cada microservicio sin estar atados a un único *stack.*
- **Comunicación a través de API:** la interacción entre los servicios se realiza mediante interfaces bien definidas, lo que reduce las dependencias.
- **Despliegue independiente:** los servicios pueden ser desarrollados, probados y desplegados de forma autónoma. Esto permite actualizaciones rápidas y continuas de una parte de la aplicación sin afectar al resto.

© Universidad Internacional de La Rioja (UNIR)

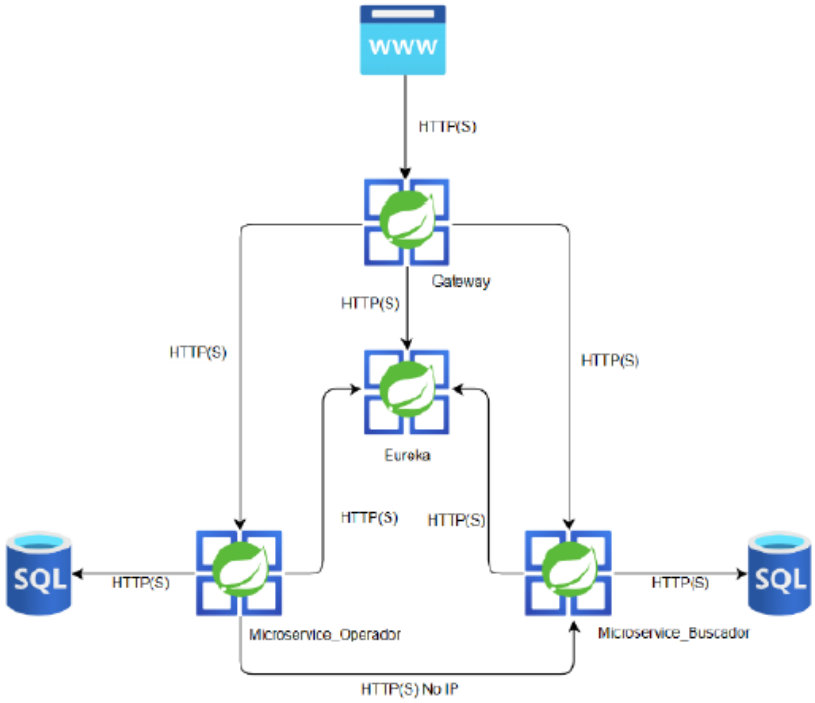


Figura 6. Arquitectura microservicios *spring*. fuente: elaboración propia.

Ventajas:

- **Escalabilidad:** es más fácil escalar un servicio específico que tiene una alta demanda en lugar de toda la aplicación.
- **Agilidad:** los equipos pueden trabajar de forma independiente, lo que acelera el desarrollo y la entrega de nuevas funcionalidades.
- **Resiliencia:** si un servicio falla, el resto de la aplicación puede seguir funcionando.

Desventajas:

- **Complejidad:** la gestión de un gran número de servicios, la comunicación entre ellos y la supervisión del sistema en su totalidad puede ser más compleja.

© Universidad Internacional de La Rioja (UNIR)

- **Coste operacional:** la infraestructura necesaria para desplegar y gestionar microservicios (contenedores, orquestadores, como Kubernetes) puede ser más costosa.

- **Latencia de red:** la comunicación a través de la red entre servicios puede introducir latencia, en comparación con las llamadas internas de una arquitectura monolítica.

Patrón de diseño MVVM

El patrón de diseño MVVM *(Model-View-ViewModel)* se utiliza, principalmente, en el desarrollo de aplicaciones con interfaces de usuario ricas en SPA3 y está muy presente en plataformas, como WPF, Xamarin, Vue y Angular (Fuksa, 2024). Otro patrón de diseño más tradicional es **MVC,** el cual es más usado en aplicaciones de **arquitectura clásica,** aunque también se puede usar en aplicaciones SPA.

Plataforma /lenguaje	*Framework*/biblioteca	Descripción breve
.NET / C#	WPF (Windows *presentation foundation*)	*Framework* de Microsoft para aplicaciones de escritorio; MVVM es el patrón recomendado.
	Xamarin.Forms / .NET MAUI	Para aplicaciones móviles multiplataforma; utiliza MVVM como patrón estándar.
JavaScript / TypeScript	Knockout.js	Librería ligera para MVVM en aplicaciones web; usa *data-binding* declarativo.
	Vue.js	Aunque se basa en MVVM, también permite patrones flexibles, como MVC o MV*.
	Angular	Soporta patrones similares a MVVM; el ViewModel se refleja en los componentes.
iOS / Swift	SwiftUI	Apple fomenta un enfoque MVVM con SwiftUI, separando lógica de UI en ViewModels.
Android / Kotlin	Android Architecture Components	Jetpack ViewModel + LiveData permiten implementar MVVM de forma eficiente.
C++ / Qt	Qt Quick (QML + C++)	Usa un enfoque MVVM con QML, como View y C++ y como Model/ViewModel.
Python	Kivy	*Framework* para aplicaciones GUI y móviles en Python. Puede adaptarse a MVVM.

Tabla 1. MVVM y *frameworks* de desarrollo. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

En este patrón, la **distribución de responsabilidades** se da de la siguiente manera:

Frontend:

El *frontend* está donde ocurre la interacción con el **usuario** y la mayor parte del **comportamiento visual** de la aplicación. Aquí, se encuentran los siguientes componentes de MVVM:

- **View:**
  - La Vista es la interfaz de usuario, es decir, todo lo que el usuario puede ver y con lo que puede interactuar.
  - En el *frontend,* es responsable de mostrar los datos que recibe del ViewModel y de enviar eventos de usuario (clics, teclas, etc.) al ViewModel para que los maneje.
  - En *frameworks,* como Angular, React, o WPF, esto podría ser representado por componentes visuales o plantillas (HTML, XAML, etc.).
- ViewModel:
  - El ViewModel actúa como un intermediario entre la Vista y el Modelo.
  - Su responsabilidad principal es transformar los datos del Modelo en un formato adecuado para la Vista y gestionar la lógica de presentación.
  - El ViewModel en el *frontend* es responsable de la lógica de la UI, como formatear datos o gestionar el estado de la vista (por ejemplo, si una ventana está activa, si un botón debe ser habilitado o no, etc.).
  - Usualmente, se encuentra vinculado a la Vista mediante un sistema de *data binding.*

© Universidad Internacional de La Rioja (UNIR)

Backend:

El *backend* maneja la lógica de negocio y los datos persistentes. En el patrón MVVM, en el *backend* se encuentran, principalmente, los siguientes componentes:

- Model (Modelo):
  - El Modelo es donde reside la lógica de negocio y la estructura de los datos.
  - Puede incluir objetos de dominio, entidades, validaciones y funciones de acceso a la base de datos.
  - En el *backend,* el Modelo es responsable de la manipulación de datos y de ejecutar las operaciones necesarias para interactuar con la base de datos (CRUD: crear, leer, actualizar, eliminar).
  - El Modelo puede ser un conjunto de clases que representan las entidades que la aplicación manipula.

Interacción entre *frontend* y *backend:*

- ***Frontend**:* La Vista (View) se comunica con el ViewModel que, a su vez, puede hacer peticiones al *backend* (Modelo).
- **Backend**: el Modelo en el *backend* puede exponer servicios o API (como RESTful API o GraphQL) que permiten al ViewModel en el *frontend* interactuar con los datos y la lógica de negocio.

© Universidad Internacional de La Rioja (UNIR)

Por ejemplo:

- El ViewModel puede realizar una solicitud HTTP a una API en el *backend* para **obtener datos** o **enviar actualizaciones.**
- El *backend* **procesa** esta solicitud en el Modelo (que puede involucrar consultar una base de datos o realizar cálculos complejos) y luego devuelve los datos al ViewModel.
- Finalmente, el ViewModel **transforma** esos datos y los **presenta** a la Vista en un formato adecuado.

Resumen:

- *Frontend:*
  - **View (Vista)**: interfaz de usuario (UI).
  - **ViewModel**: lógica de presentación. Es el mediador entre la Vista y el Modelo.
- *Backend:*
  - **Model (Modelo)**: lógica de negocio, manipulación de datos, interacción con base de datos.

El patrón MVVM ayuda a separar la lógica de presentación de la lógica de negocio y hace que el código sea más **mantenible,** especialmente en aplicaciones con interfaces complejas.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Con respecto a la **seguridad,** es necesario mitigar vulnerabilidades mediante la **autenticación, autorización, validación** de entrada para asegurar la integridad y confidencialidad de la información de los datos alojados en la aplicación y en la base de datos. Asimismo, hay que prestar atención al diseño de un **registro seguro** (los mensajes que deben aparecer deben ser los que realmente aportan información útil para los auditores de seguridad) y **manejo de errores** y **excepciones,** de forma que no aporten a los usuarios más información de la necesaria, ya que podría ser útil a los atacantes para sacar provecho de ella. La información de errores completa solo debe estar accesible para administradores y auditores de seguridad.

En resumen, las vulnerabilidades de seguridad debidas a la arquitectura de diseño física y lógica son causadas por **defectos** en los servicios de autenticación, autorización, sesiones, integridad y confidencialidad de datos, trazabilidad, gestión de errores y excepciones y comunicación entre capas.

© Universidad Internacional de La Rioja (UNIR)

2.3. Protocolo HTTP

El protocolo HTTP no fue diseñado para aplicaciones y probablemente tampoco para usos seguros. HTTP crea oportunidades para generar vulnerabilidades de seguridad. El protocolo HTTP implementa una serie de métodos especificados en **HTTP 1.1 rfc 2616**, según la página web de datatracker (2020) y (Mozilla, s.f.), que son los siguientes:

- **HEAD:** pide una respuesta idéntica a la que correspondería a una petición GET, pero sin el cuerpo de la respuesta. Esto es útil para la recuperación de metainformación escrita en los encabezados de respuesta, sin tener que transportar todo el contenido.

- **GET:** pide una representación del recurso especificado. Por seguridad no debería ser usado por aplicaciones que causen efectos, ya que transmite información a través de la URI agregando parámetros a la URL.

> Ejemplo
> GET /images/logo.png HTTP/1.1 obtiene un recurso llamado logo.png
>
> Ejemplo con parámetros
> GET /index.php?page=main&lang=es

- **POST:** envía los datos para que sean procesados por el recurso identificado (URL). Estos se incluirán en el cuerpo de la petición. Esto puede resultar en la creación de un nuevo recurso o de las actualizaciones de los recursos existentes o en ambas cosas.

© Universidad Internacional de La Rioja (UNIR)

- **PUT:** sube, carga o realiza un *upload* de un recurso especificado (archivo). Es el camino más eficiente para subir archivos a un servidor. Esto es porque en POST utiliza un mensaje multiparte y es decodificado por el servidor. En contraste, el método PUT te permite escribir un archivo en una conexión *socket* establecida con el servidor. La desventaja del método PUT es que los servidores de *hosting* compartido no lo tienen habilitado.

> Ejemplo
> PUT /path/filename.html HTTP/1.1.

- **DELETE:** borra el recurso especificado.

- **TRACE:** este método solicita al servidor que envíe de vuelta en un mensaje de respuesta. Se utiliza con fines de comprobación y diagnóstico. El método *trace* da lugar a la vulnerabilidad Cross Site Tracing (XST) debido a que copia las cabeceras de la petición en el cuerpo de la respuesta haciendo inútil la protección mediante el parámetro HTTPONLY que impide acceso a la *cookie* que protege. XST se puede combinar con un ataque XSS para robo de un identificador de sesión contenido en una *cookie.*

© Universidad Internacional de La Rioja (UNIR)

```
$ telnet foo.com 80
Trying 127.0.0.1...
Connected to foo.bar.
Escape character is '^]'.
TRACE / HTTP/1.1
Host: foo.bar
X-Header: test

HTTP/1.1 200 OK
Date: Mon, 02 Dec 2002 19:24:51 GMT
Server: Apache/2.0.40 (Unix)
Content-Type: message/http

TRACE / HTTP/1.1
Host: foo.bar
X-Header: test
```

```
<script type="text/javascript">
<!--
function sendTrace () {
    var xmlHttp = new ActiveXObject("Microsoft.XMLHTTP");
    xmlHttp.open("TRACE", "http://foo.bar",false);
    xmlHttp.send();
    xmlDoc=xmlHttp.responseText;
    alert(xmlDoc);
}
//-->
</script>

<INPUT TYPE=BUTTON OnClick="sendTrace();" VALUE="Send Trace Request">
```

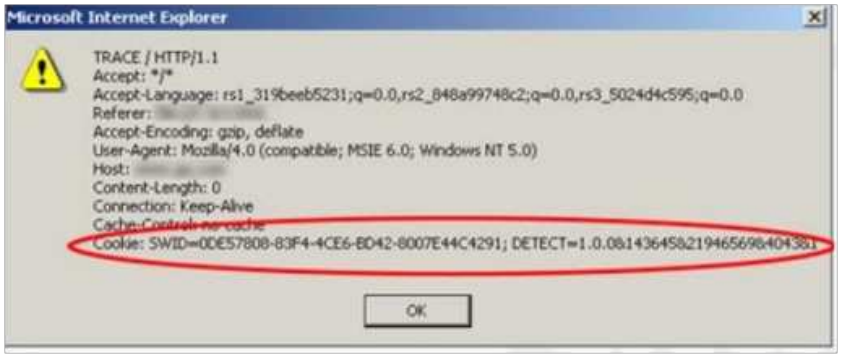


Figura 7. XST. Fuente: adaptado de Grossman, 2003.

- **OPTIONS:** devuelve los métodos HTTP que el servidor soporta para un URL específico. Este método puede ser utilizado para comprobar la funcionalidad de un servidor web.

- **CONNECT:** se utiliza para saber si se tiene acceso a un *host*. No necesariamente la petición llega al servidor. Este método se utiliza principalmente para saber si un proxy nos da acceso a un *host*.

© Universidad Internacional de La Rioja (UNIR)

En la Figura 8 se observa cómo se puede especificar un método HTTP.

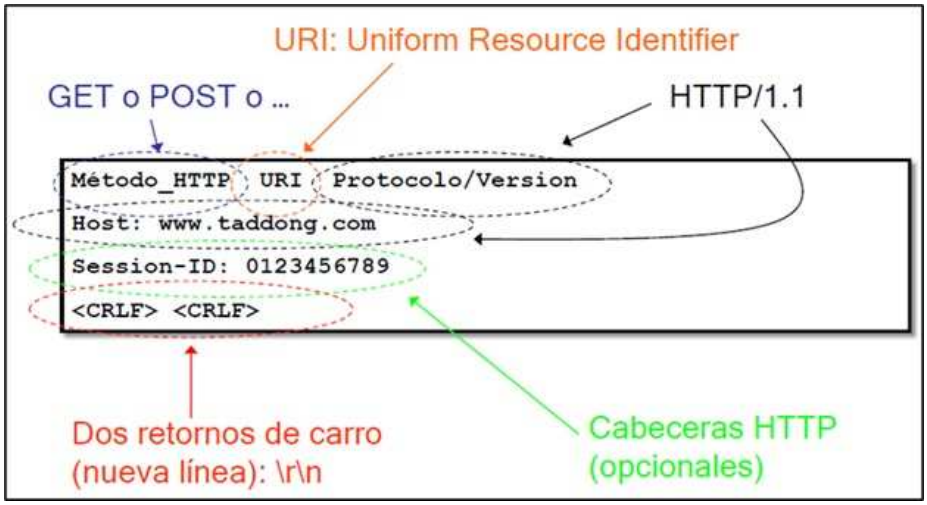


Figura 8. Métodos HTTP. Fuente: elaboración propia.

En una petición GET los parámetros que se incluyen en la cadena de la URL pueden ser rutinariamente escritos a LOG cache del navegador y almacenados en el historial revelando información sensible, como *passwords,* entre otras. Es más sencillo prevenir XSS deshabilitando GET en las aplicaciones. Para un atacante es más fácil enviar correos con una URL con parámetros maliciosos:

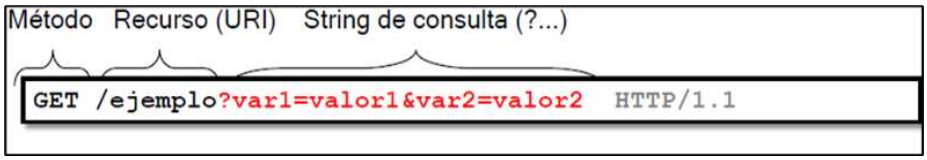


Figura 9. Método HTTP GET. Fuente: elaboración propia.

Sin embargo, con el método POST los parámetros se especifican en el cuerpo de la petición. Para que sea más seguro se debe de **utilizar POST en lugar de GET.**

© Universidad Internacional de La Rioja (UNIR)

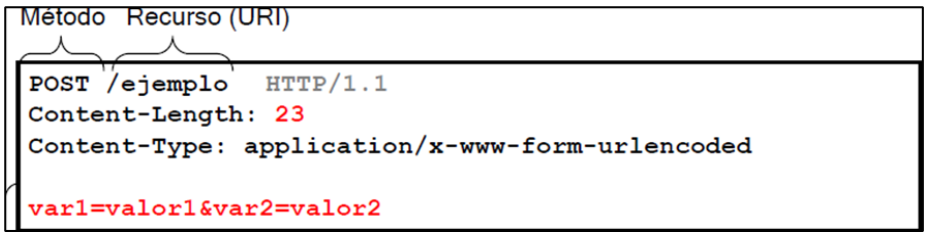


Figura 10. Método HTTP POST. Fuente: elaboración propia.

Procedencia de peticiones: si la aplicación usa un identificador de sesión *(cookie)* con un formulario (administración) para crear un usuario:

```
<form method="POST" action="/new_user" >
  Name of new user: <input type="text" name="username">
  Password for new user: <input type="password" name="user_passwd">
    <input type="submit" name="action" value="Create User">
</form>
```

Figura 11. Procedencia de peticiones CSRF. Fuente: Chess y West, 2007.

Un atacante podría tener un sitio web con:

```
<form method="POST" action="http://www.example.com/new_user">
  <input type="hidden" name="username" value="hacker">
  <input type="hidden" name="user_passwd" value="hacked">
</form>
<script>
  document.usr_form.submit();
</script>
```

Figura 12. Procedencia de peticiones CSRF. Fuente: Chess y West, 2007.

Si el administrador visita el sitio controlado por el atacante se puede dar un ataque Cross Site Request Forguery (CSRF). Para prevenirlo se puede incluir algún campo que la aplicación pueda validar:

© Universidad Internacional de La Rioja (UNIR)

```
<form method="POST" action="/new_user" >
  Name of new user: <input type="text" name="username">
  Password for new user: <input type="password"
name="user_passwd">
    <input type="submit" name="action" value="Create User">
    <input type="hidden" name="req_id" value="87ae34d92ba7a1">
</form>
```

Figura 13. Procedencia de peticiones. Protección CSRF. Fuente: Chess y West, 2007.

Otra vulnerabilidad que se puede presentar en el protocolo HTTP es la **Http *Response Spliting*** que consiste en incluir caracteres **\r\n** de retorno de carro y de línea para generar una respuesta doble (https://www.acunetix.com/websitesecurity/crlf-injection/) que se ejecutarán en el navegador del usuario permitiendo lanzar otros ataques como XSS, que puede verse en la Figura 14.

```
String author = request.getParameter("author");
Cookie cookie = new Cookie("author", author);
cookie.setMaxAge(cookieExpiration);
response.addCookie(cookie);
```

```
HTTP/1.1 200 OK
Set-Cookie: author=Jane Smith
...
```

Figura 14. HTTP Response Splitting. Fuente: Chess y West, 2007.

Si el atacante suministra una cadena como **Wiley Hacker\r\n\r\nHTTP/1.1 200 OK\r\n…** la respuesta se convierte en doble permitiendo inyectar código HTML:

```
HTTP/1.1 200 OK
Set-Cookie: author=Wiley Hacker

HTTP/1.1 200 OK
...
```

Figura 15. HTTP Response Splitting. Fuente: Chess y West, 2007.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

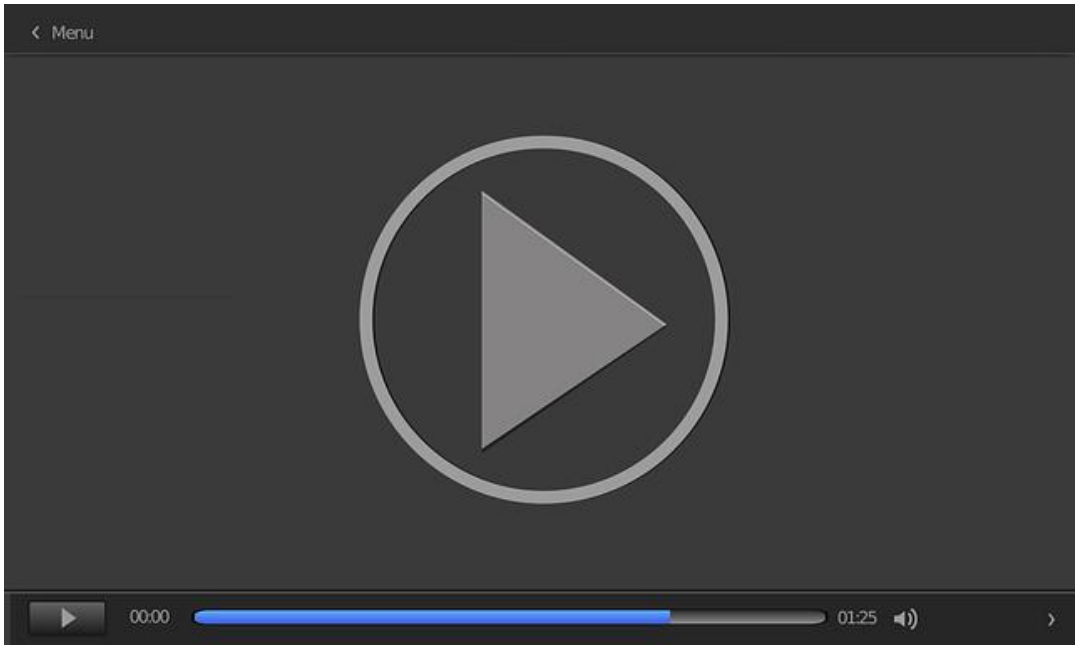
El problema del protocolo HTTP es más agudo cuando se maneja **el estado de sesión,** que es necesario en la mayor parte de aplicaciones porque el protocolo en sí mismo es sin estado. Como **HTTP es sin estado**, construir casi cualquier tipo de aplicación sofisticada requiere un identificador de sesión que sirva para ambos sentidos de la comunicación para asociar las peticiones previas de un usuario con las siguientes. Los identificadores de sesión pueden ser pasados hacia adelante y hacia atrás como parámetros URL. Actualmente, la mayor parte de las aplicaciones manejan **cookies**, que es un mecanismo que el protocolo incorporó posteriormente para almacenar datos en la memoria del navegador con un espacio de 4096 bytes. La razón más común de usar un identificador de sesión es permitir a un usuario autenticarse solo una vez para continuar con una serie de interacciones con la aplicación. Se puede utilizar una cookie, además de otros métodos, para almacenar el identificador de sesión.

Esto quiere decir que la seguridad de la aplicación depende de ello y debe ser muy difícil para un atacante averiguar u obtener el **identificador de sesión** para un usuario autenticado. Una buena gestión de sesión HTTP escoge identificadores de sesión fuertes y se asegura de que son publicados y revocados en puntos apropiados en el programa.

© Universidad Internacional de La Rioja (UNIR)

2.4. Ciclo de vida de desarrollo seguro de software (SSDLC)

En el vídeo *SSDLC* veremos qué es el ciclo de vida de desarrollo seguro de *software* (SSDLC), sus fases y actividades.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=6c1c2155-6150-4195-bda5-adbf010de7a9

Integrar la seguridad en cada fase del ciclo de vida del desarrollo de *software* es fundamental para construir aplicaciones web robustas. Este enfoque, a menudo denominado DevSecOps, garantiza que la **seguridad** sea una **responsabilidad compartida** y no una ocurrencia tardía. Uno de los modelos SSDLC abiertos es OWASP SAMM (Figura 16) (OWASP-SAMM, s. f.).

© Universidad Internacional de La Rioja (UNIR)



Figura 16. OWASP SAMM. Fuente: Software Assurance Maturity Model, s. f.

Para **metodologías ágiles,** se puede consultar este trabajo de aplicación de seguridad en El ciclo de vida desarrollo (De Vicente et al. 2019). Otro documento de referencia es la guía del desarrollador OWASP (OWASP DEV, s. f.). Para ayuda en la gestión de la implementación y puesta en marcha de un proceso SSDLC, se dispone de la herramienta SAMMY (SAMMY, s. f.).

Diseño y modelado de amenazas

Durante la fase de diseño, se debe realizar un modelado de amenazas para identificar posibles **vulnerabilidades** en la arquitectura de la aplicación. El modelado de amenazas, junto con el análisis de riesgos de los activos de la organización, permiten implementar obtener los requisitos de seguridad desde el principio.

Requisitos y arquitectura de seguridad

En la fase inicial de planificación, es crucial definir los requisitos de seguridad de la aplicación. Esto incluye identificar los **tipos de datos** que se manejarán, los **roles** de los usuarios y sus permisos y los requisitos de **cumplimiento normativo** (como GDPR o HIPAA).

© Universidad Internacional de La Rioja (UNIR)

Desarrollo seguro

En la fase de desarrollo, los programadores deben seguir prácticas de **codificación** segura. Esto incluye el uso de *frameworks* de desarrollo seguro, la validación de todas las entradas del usuario, el uso de consultas parametrizadas para evitar la inyección de SQL y la codificación de las salidas para prevenir ataques de *Cross-Site Scripting* (XSS).

Pruebas de seguridad

Antes del despliegue en producción, la aplicación debe someterse a rigurosas **pruebas de seguridad.** Esto incluye:

- **Análisis estático de seguridad de aplicaciones (SAST):** herramientas que analizan el código fuente en busca de vulnerabilidades. Fase de desarrollo.
- **Análisis dinámico de seguridad de aplicaciones (DAST):** herramientas que prueban la aplicación en ejecución en busca de vulnerabilidades. Fase de pruebas.
- **Análisis interactivo de seguridad de aplicaciones (IAST)**: herramientas que, de forma interactiva con la aplicación Y en combinación con el uso de herramientas DAST, encuentran vulnerabilidades en la aplicación. Fase de pruebas.
- **Análisis de seguridad de las dependencias (SCA)**: herramientas que encuentras vulnerabilidades CVE en las dependencias de terceros. Fase de desarrollo.
- **Pruebas funcionales de seguridad:** utilizan herramientas de diversos tipos y prueban la robustez de implementación de cada uno de los requisitos de seguridad. Fase de pruebas.

© Universidad Internacional de La Rioja (UNIR)

Despliegue y mantenimiento

Una vez desplegada, la aplicación debe ser **monitoreada continuamente** en busca de actividades sospechosas. Es crucial mantener todos los componentes de *software* actualizados con los últimos **parches** de seguridad y tener un **plan de respuesta** a incidentes en caso de una brecha de seguridad.

© Universidad Internacional de La Rioja (UNIR)

2.5. Referencias bibliográficas

Asadullah, S., Hussaini, S. y Khader, M. (2020). AJAX Architecture Implementation Techniques. *International Journal of Engineering Research and Applications (IJERA), 3*(2), 111-117. https://www.ijera.com/papers/Vol3_issue2/O32111117.pdf

Chess, B. y West, J. (2007). *Secure programming with static analysis.* Addison-Wesley.

El Akhdar, A., Baidada, C., Kartit, A., Hanine, M., García, C. O., Lara, R. G. y Ashraf, I. (2024). Exploring the potential of microservices in internet of things: a systematic review of security and prospects. *Sensors*, *24*(20), 6771. https://www.mdpi.com/1424-8220/24/20/6771

Fuksa, M., Speth, S. y Becker, S. (2024). MVVM Revisited: Exploring design variants of the Model-View-ViewModel Pattern. In *International Conference on Enterprise Design, Operations, and Computing* (pp. 163-181). Springer Nature Switzerland. https://arxiv.org/pdf/2504.18191

Grossman, J. (2003). *XST* [imagen]. https://www.cgisecurity.com/whitehat-mirror/WH-WhitePaper_XST_ebook.pdf

Haindi, P., Kochberger, P. y Sveggen, M. (2024). A systematic literature review of inter-service security threats and mitigation strategies in microservice architectures. *IEEE Access*, *12*, 90252- 90286. https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10540127

Mayekar, B. (2018). *Explain in detail AJAX Web Application Model with neat diagram.* Q u e s 1 0 . https://www.ques10.com/p/29472/explain-in-detail-ajax-web-application-model-wit-1/

© Universidad Internacional de La Rioja (UNIR)

Mcgraw, G. (2006). *Seven Touchpoints for Software Security* [figura]. http://www.swsec.com/resources/touchpoints/

Mozilla HTTP. Protocolo HTTP. Disponible en: https://developer.mozilla.org/es/docs/Web/HTTP

Nguyen, A. (2024). *What are progressive web apps: Architecture & benefits explained.* Synodus. https://synodus.com/blog/web-development/progressive-web-apps/

OWASP SAMM. *Modelo de ciclo de vida de desarrollo seguro.* OWASP. https://owasp.org/www-project-samm/

Página de Datacracker (https://datatracker.ietf.org/doc/html/rfc2616).

Software Assurance Maturity Model. (s. f.). https://owasp.org/www-project-samm/

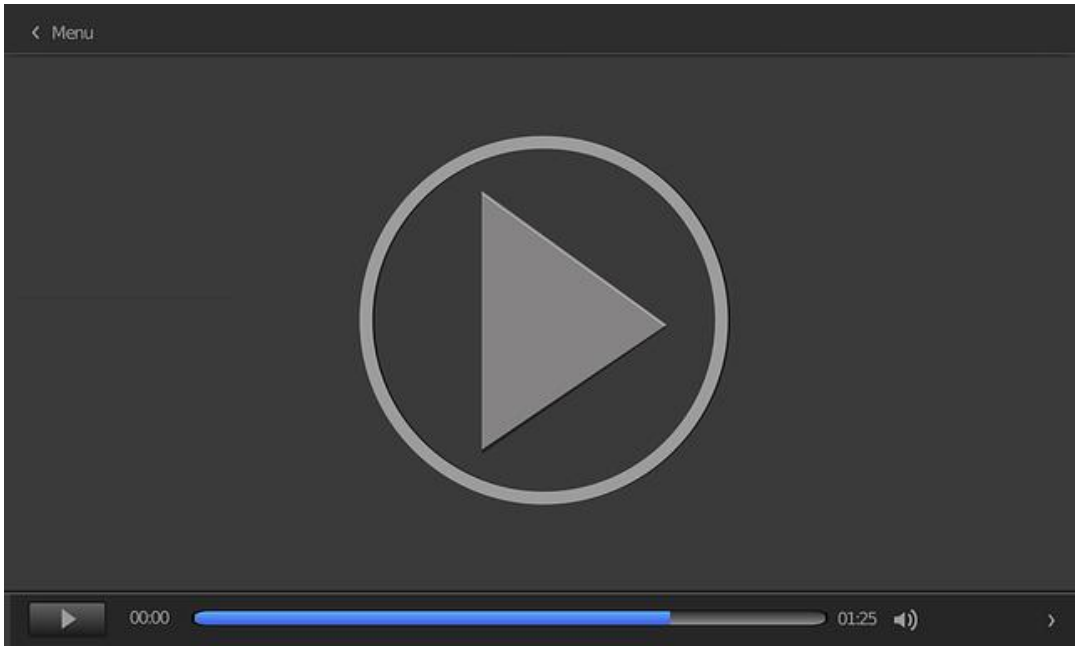
Wang, M. y Gu, G. (2025). *GuardianPWA: Enhancing Security Throughout the Progressive Web App Installation Lifecycle.* https://arxiv.org/pdf/2509.13561

© Universidad Internacional de La Rioja (UNIR)

Descripción del modelo de arquitectura de aplicaciones SPA AJAX

iDESWEB UA. (2013, noviembre 17). *Ejecución de una aplicación web: modelo moderno (AJAX)* [vídeo]. YouTube. https://www.youtube.com/watch?v=l3dmjFMefaE

Descripción del modelo de arquitectura de aplicaciones web SPA AJAX.



Ejecución de una aplicación web: modelo moderno (AJAX)

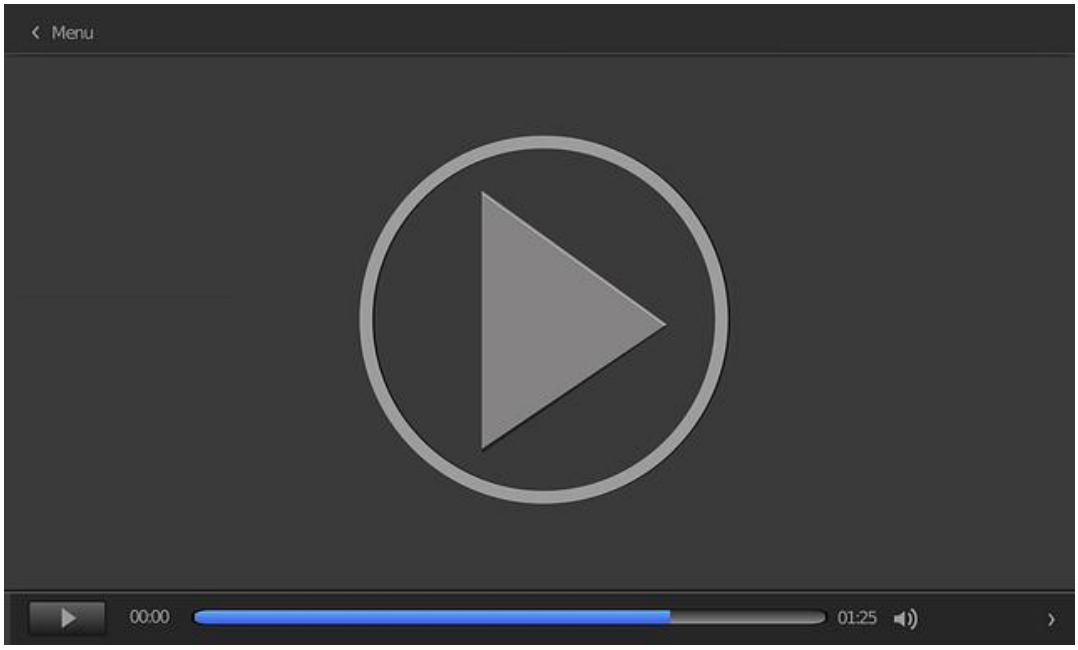
Accede al vídeo:

https://www.youtube.com/embed/l3dmjFMefaE

© Universidad Internacional de La Rioja (UNIR)

Arquitectura de microservicios

Uncle Dave's Code. (2023, junio 24). *Creación de una arquitectura de microservicios* [vídeo]. YouTube. https://www.youtube.com/watch?v=NO9qO0Z6GdY



Creando una Aplicación Escalable - Microservicios con Spring Boot

Accede al vídeo:

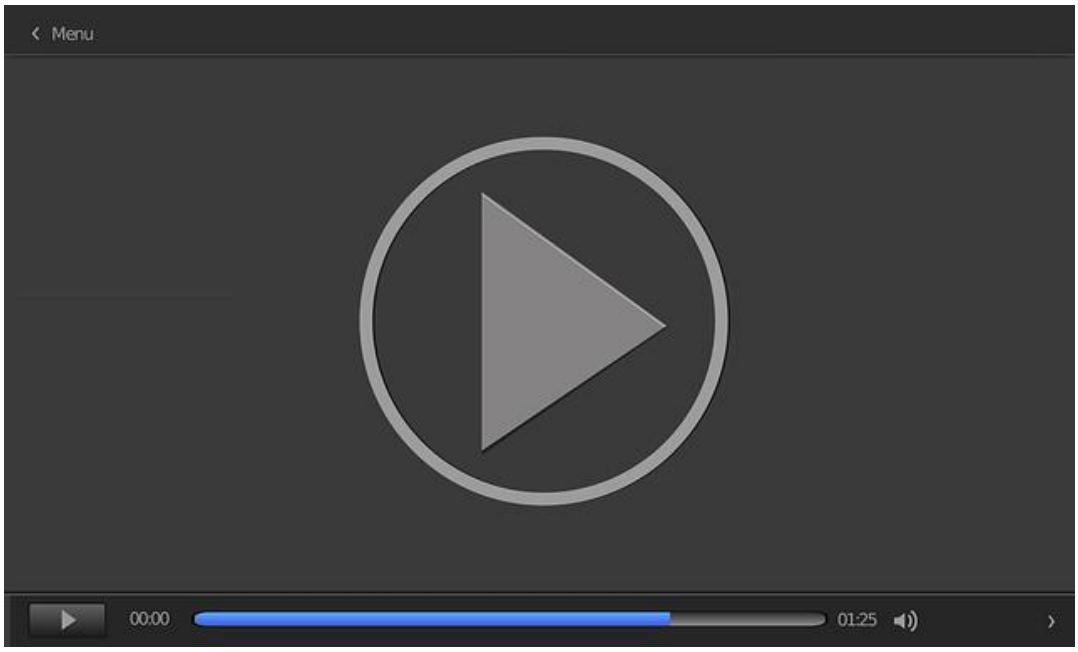
https://www.youtube.com/embed/NO9qO0Z6GdY

© Universidad Internacional de La Rioja (UNIR)

Mean stack

Conde, J. (2014, octubre 21). *48.- Curso de AngularJS. Introducción al trabajo con el "MEAN STACK"* [vídeo]. YouTube. https://www.youtube.com/watch?v=F_Mi3WHPrDw

MEAN.JS es una solución JavaScript completa que ayuda a construir aplicaciones web de producción rápidas, robustas y mantenibles usando MongoDB, Express, AngularJS y Node.js. Es un ejemplo de arquitectura SPA.



48.- Curso de AngularJS. Introducción al trabajo con el "MEAN STACK"

Accede al vídeo:

https://www.youtube.com/embed/F_Mi3WHPrDw

© Universidad Internacional de La Rioja (UNIR)

Patrón de diseño MVVM

Stonis, M. (2022). *Enterprise application patterns using .NET MAUI.* Microsoft. https://learn.microsoft.com/es-es/dotnet/architecture/maui/mvvm

El patrón MVVM ayuda a separar limpiamente la lógica de presentación y negocios de una aplicación de su interfaz de usuario (UI). Mantener una separación limpia entre la lógica de la aplicación y la interfaz de usuario ayuda a abordar numerosos problemas de desarrollo y facilita la prueba, el mantenimiento y la evolución de una aplicación. También, puede mejorar considerablemente las oportunidades de reutilización del código y permite a los desarrolladores y a los diseñadores de la interfaz de usuario colaborar más fácilmente al desarrollar sus respectivas partes de una aplicación.

© Universidad Internacional de La Rioja (UNIR)

The Application of a new secure software development Life Cycle (S-SDLC) with agile methodologies

De Vicente Mohino, J., Bermejo Higuera, J., Bermejo Higuera, J. R. y Sicilia Montalvo, J. A. (2019). The application of a new secure software development Life Cycle (S-SDLC) with agile methodologies. *Electronics, 8*(11), 1218. https://www.mdpi.com/2079-9292/8/11/1218

Hoy en día, los modelos ágiles están en auge. Se definen por la forma en que logran la interacción e integración de todos los involucrados en el ciclo de vida del *software,* las ventajas de la rápida reacción al cambio y la implementación de artefactos o entregables que muestran el nivel de progreso alcanzado en cualquier momento. En este contexto, parece claramente necesario definir un nuevo modelo de desarrollo de *software* que dé prioridad a los aspectos de seguridad en cualquier fase del ciclo de vida de este y aproveche los beneficios de los modelos ágiles. La metodología propuesta muestra que si se considera la seguridad desde el principio, las vulnerabilidades se detectan y resuelven fácilmente durante el tiempo previsto para el proyecto, sin tiempo ni costos adicionales para el cliente.

© Universidad Internacional de La Rioja (UNIR)

Test

1. ¿Qué método HTTP se usa para eliminar un recurso en el servidor?

   A. GET.

   B. POST.

   C. DELETE.

   D. PUT.

2. ¿Cuál de los siguientes es un modelo de desarrollo de *software* seguro?

   A. OPEN SAMM.

   B. CSF.

   C. ISO 27001.

   D. Todos las anteriores son ciertas.

3. ¿En qué fase se debe realizar una prueba de penetración a una aplicación web?

   A. Diseño.

   B. Desarrollo.

   C. Pruebas.

   D. Producción.

4. ¿De qué depende la derivación de requisitos de seguridad?

   A. El modelado de amenazas o casos de abuso.

   B. Del análisis de riesgos.

   C. Análisis de seguridad del código fuente.

   D. La A y la B.

© Universidad Internacional de La Rioja (UNIR)

5. Antes de la prueba de penetración, ¿qué actividades de seguridad hay que llevar a cabo?

A. Derivación de requisitos de seguridad.

B. Modelado de amenazas.

C. Análisis de seguridad del código fuente.

D. Todas las anteriores son ciertas.

6. ¿Cuál es la norma, modelo o proceso de seguridad que debe adoptarse para implementar la seguridad de una aplicación web desde el principio del desarrollo?

A. SSDLC.

B. SGSI.

C. SQLI.

D. OWASP.

7. ¿Qué método HTTP permite conectarse a un servicio situado detrás de un proxy?

A. PUT.

B. GET.

C. POST.

D. CONNECT.

8. ¿Qué método HTTP permite obtener solo las cabeceras de la aplicación web?

A. PUT

B. HEAD

C. POST

D. GET

© Universidad Internacional de La Rioja (UNIR)

9. ¿Qué método HTTP envía los parámetros en la URL?

A. GET.

B. POST.

C. PUT.

D. Todas las anteriores.

10. ¿Qué puede suponer la vulnerabilidad de HTTP *response splitting*?

A. La B y la D son correctas.

B. La inyección de código JavaScript.

C. La inyección de código SQL.

D. La inyección de código HTML.

© Universidad Internacional de La Rioja (UNIR)